

Laboratorio di Ingegneria Informatica – Progetto Finale:

Pipeline di Parsing Web per il Chatbot Minerva

Matteo Martin Console

Matricola: 2112194

console.2112194@studenti.uniroma1.it

Alice Agneni

Matricola: 2128893

agneni.2128893@studenti.uniroma1.it

Giulia Codella

Matricola: 2118019

codella.2118019@studenti.uniroma1.it

1 Obiettivo del Progetto

Il progetto realizza una pipeline end-to-end per l'acquisizione e l'analisi di documenti provenienti da sorgenti web eterogenee, a supporto dell'LLM nazionale **Minerva** (Sapienza NLP / Babelscape). Dato un URL appartenente a uno dei domini supportati, il sistema scarica la pagina, ne estrae il testo informativo pulito in Markdown eliminando il rumore tipico delle pagine web, valuta la qualità del testo estratto confrontandolo con un Gold Standard sia tramite metriche quantitative sia tramite un LLM Judge, persiste tutti i dati in un database relazionale ed espone l'intera funzionalità tramite REST API e Web UI, coordinati in un'architettura multi-container Docker Compose.

2 Organizzazione del Codice

Il progetto si articola in due microservizi separati (backend e frontend), ciascuno con il proprio Dockerfile e requirements.txt, coordinati da un docker-compose.yaml che orchestra anche **MariaDB** e **Ollama**, montando come volumi condivisi gs_data/, domains.json e i dati persistenti di database e LLM.

Backend (backend/src/server.py). Implementato con **FastAPI** (porta 8003) e validazione I/O tramite **Pydantic**, con inizializzazione del database gestita nel ciclo di vita dell'applicazione. Espone tutti gli endpoint richiesti dalla specifica: GET /domains, POST /parse (con flag local per il parsing da HTML già salvato), GET /gold_standard, GET /gold_standard_urls, POST /evaluate, POST /evaluate_judge, GET /full_gs_eval, POST /add_web_resource, POST /add_gold_standard, DELETE /web_resource, DELETE /gold_standard, GET /db_stats, GET /db_schema e GET /status. La funzione interna get_parsed_data instrada la richiesta al parser corretto in base

al netloc dell'URL; gli errori (dominio non supportato, risorsa assente nel DB) sono sollevati come HTTPException. La normalizzazione del Markdown prima del calcolo delle metriche avviene tramite remove_markdown (mistune + BeautifulSoup).

Parser dedicati (backend/src/logic/). Sono implementati quattro parser asincroni, uno per dominio assegnato, basati su **Crawl4AI** con BrowserConfig headless, in grado di gestire sia il crawling diretto da URL sia il parsing di HTML grezzo passato tramite POST /parse (scritto in un file temporaneo, letto con prefisso file://). parser_wikipedia.py isola il corpo dell'articolo con il selettore #mw-content-text e uno script iniettato che rimuove note, infobox, navbox e la sezione "See also". parser_grammy.py usa il selettore main escludendo header, footer, nav, aside, script e form, con titolo ricavato dai metadati o dal primo heading. parser_huddle.py isola article e applica clean_huddle_markdown, che rimuove i link mantenendo il testo e scarta righe con termini da blacklist o più corte di 30 caratteri. parser_academia.py, per le protezioni anti-bot, usa uno User-Agent reale e un delay prima dell'estrazione, con cascata: div abstract/summary via regex, poi Markdown filtrato, infine meta tag og:description.

Metriche (backend/src/logic/metrics.py). compute_token_level_eval implementa la token_level_eval obbligatoria: i token sono estratti con la regex [a-z0-9]+ sul testo in minuscolo, formando gli insiemi tokens_estratti e tokens_gs; precision, recall e F1 sono calcolati sulla loro intersezione, con gestione degli insiemi vuoti.

Persistenza (database.py, models.py, init_db.py). L'accesso al database avviene tramite **SQLAlchemy** con connettore

mariadb+mariadbconnector. Sono definiti quattro modelli ORM: WebResource e GoldStandard (tabelle obbligatorie, in relazione uno-a-uno con cancellazione a cascata), Evaluation e JudgeEvaluation, che memorizzano metriche e giudizi del Judge pre-calcolati per URL. All'avvio, `init_db()` crea lo schema e popola le tabelle leggendo i file JSON in `gs_data/`, evitando duplicati.

LLM Judge (llm_judge.py). `evaluate_with_llm` invia a **Ollama** (modello `llama3.2:3b`, formato JSON forzato) un prompt che richiede punteggio 1-5 e feedback testuale, troncando i testi a 2000 caratteri, con fallback in caso di errore. Il giudizio è invocato da `POST /evaluate_judge` e in forma aggregata da `GET /full_gs_eval`, che lo salva nel database.

Frontend (frontend/src/frontend.py). Server FastAPI (porta 8004) con Jinja2, organizzato in home page e tre sezioni: `index.html` (stato del sistema e domini supportati), `evaluation.html` (test del parser su un URL in modalità Live/Local, confronto con GS, metriche e giudizio del Judge), `gs_builder.html` (download dell'HTML, inserimento manuale del gold text, gestione delle entry per dominio) e `stats.html` (tabella con conteggi e medie per dominio).

Infrastruttura. `docker-compose.yaml` avvia i quattro container con path relativi, esponendo le porte 8003, 8004, 3306 e 11434.

3 Valutazione dei Parser

Per ciascuno dei quattro domini assegnati sono stati costruiti manualmente i Gold Standard. La valutazione automatica avviene tramite `GET /full_gs_eval`, che esegue il parsing di ogni entry del GS dall'HTML statico salvato nel database, calcola la `token_level_eval` sul testo normalizzato e aggrega i risultati come media aritmetica. La tabella 1 riassume i risultati.

Dominio	Prec.	Recall	F1
en.wikipedia.org	0.9952	0.9996	0.9974
grammy.com	0.9954	0.9937	0.9945
www.academia.edu	0.9866	1.000	0.9932
www.huddle.org	1.000	0.9997	0.9998

Table 1: Risultati medi della `token_level_eval` sui Gold Standard.

Tutti i parser si collocano nella fascia “Buono” definita dalla specifica ($F1 > 0.80$), con un margine ampio rispetto alla soglia. Wikipedia ottiene i risultati migliori grazie alla struttura DOM regolare e al filtraggio JavaScript pre-crawling. Academia.edu, penalizzato dal paywall parziale, raggiunge comunque un F1 elevato grazie alla strategia di estrazione a cascata. Per `huddle.org` e `grammy.com`, domini con layout meno regolare, le blacklist lessicali e i filtri sui tag strutturali riducono efficacemente gli elementi pubblicitari non eliminati dal solo selettore CSS. Il modello scelto per l'LLM Judge è `llama3.2:3b` via Ollama: tra i modelli proposti è risultato il miglior compromesso tra qualità del giudizio e tempo di risposta su CPU, dato che il giudizio viene richiamato ripetutamente durante l'aggregazione di `GET /full_gs_eval`. Il prompt include testo del parser e `gold_text`, troncato a 2000 caratteri, e richiede un output JSON con punteggio intero e motivazione testuale, salvato nelle tabelle di valutazione del database.

4 Organizzazione del Database

Lo schema segue le due tabelle obbligatorie più due tabelle aggiuntive per la persistenza delle valutazioni. `web_resources` memorizza URL (chiave primaria), dominio, titolo e HTML grezzo; `gold_standard` è in relazione uno-a-uno con `web_resources` tramite foreign key sull'URL, con cancellazione a cascata, e contiene il testo di riferimento. Le tabelle `evaluations` e `judge_evaluations`, a chiave surrogata autoincrementale e foreign key verso `web_resources.url`, memorizzano rispettivamente `precision/recall/F1` e `punteggio/feedback` del Judge pre-calcolati per URL, permettendo a `GET /db_stats` di restituire conteggi e medie senza ripetere parsing e valutazione ad ogni richiesta. Il popolamento iniziale avviene tramite `init_db()`, che legge i file JSON in `gs_data/` e inserisce risorsa web e Gold Standard.